

Coding Interview Prep — Complete Practice Problem List

Based on your NeetCode roadmap screenshot, here's a detailed practice problem list for each topic. Arranged in Easy → Medium → Hard order within each topic, so confidence builds gradually.

How to use this:

Each problem has a checkbox ☐ — mark it ☒ after solving to track your progress

Search the problem name on LeetCode and you'll find it (names match LeetCode exactly)

Don't move to the next topic until you finish one — the order follows the roadmap's dependencies (Arrays → Two Pointers → Stack → ...)

1. Arrays

Core pattern: prefix sum, sorting, hashing, in-place manipulation

- ☐ Contains Duplicate
- ☐ Valid Anagram
- ☐ Two Sum
- ☐ Group Anagrams
- ☐ Top K Frequent Elements
- ☐ Product of Array Except Self
- ☐ Valid Sudoku
- ☐ Encode and Decode Strings
- ☐ Longest Consecutive Sequence
- ☐ Best Time to Buy and Sell Stock
- ☐ Maximum Subarray (Kadane's Algorithm)
- ☐ Maximum Product Subarray
- ☐ Find Minimum in Rotated Sorted Array
- ☐ Search in Rotated Sorted Array
- ☐ 3Sum
- ☐ Container With Most Water
- ☐ Rotate Array
- ☐ Merge Intervals
- ☐ Insert Interval
- ☐ Non-overlapping Intervals
- ☐ Meeting Rooms II
- ☐ Set Matrix Zeroes
- ☐ Spiral Matrix
- ☐ Rotate Image
- ☐ Word Search

2. Two Pointers

Core pattern: scanning from both ends of a sorted array/string, fast-slow pointer

- ☐ Valid Palindrome
- ☐ Two Sum II - Input Array Is Sorted
- ☐ 3Sum
- ☐ 3Sum Closest
- ☐ 4Sum
- ☐ Container With Most Water
- ☐ Trapping Rain Water
- ☐ Remove Duplicates from Sorted Array
- ☐ Move Zeroes
- ☐ Sort Colors (Dutch National Flag)
- ☐ Reverse String
- ☐ Squares of a Sorted Array
- ☐ Boats to Save People
- ☐ Linked List Cycle (fast-slow pointer)
- ☐ Middle of the Linked List

3. Stack

Core pattern: using LIFO to solve nested-structure or "matching" problems

- ☐ Valid Parentheses
- ☐ Min Stack
- ☐ Evaluate Reverse Polish Notation
- ☐ Generate Parentheses
- ☐ Daily Temperatures
- ☐ Car Fleet
- ☐ Largest Rectangle in Histogram
- ☐ Next Greater Element I & II
- ☐ Simplify Path
- ☐ Decode String
- ☐ Asteroid Collision
- ☐ Basic Calculator II
- ☐ Remove All Adjacent Duplicates in String
- ☐ Online Stock Span
- ☐ Implement Queue using Stacks

4. Linked List

Core pattern: pointer manipulation, dummy node, reversal, cycle detection

- ☐ Reverse Linked List
- ☐ Merge Two Sorted Lists
- ☐ Reorder List
- ☐ Remove Nth Node From End of List
- ☐ Copy List with Random Pointer
- ☐ Add Two Numbers
- ☐ Linked List Cycle
- ☐ Find the Duplicate Number
- ☐ LRU Cache
- ☐ Merge K Sorted Lists
- ☐ Reverse Nodes in k-Group
- ☐ Sort List
- ☐ Palindrome Linked List
- ☐ Intersection of Two Linked Lists
- ☐ Flatten a Multilevel Doubly Linked List
- ☐ Swap Nodes in Pairs
- ☐ Rotate List
- ☐ Design Linked List

5. Binary Search

Core pattern: $O(\log n)$ search over a sorted space, narrowing the search space

- ☐ Binary Search
- ☐ Search a 2D Matrix
- ☐ Koko Eating Bananas
- ☐ Find Minimum in Rotated Sorted Array
- ☐ Search in Rotated Sorted Array
- ☐ Search in Rotated Sorted Array II
- ☐ Time Based Key-Value Store
- ☐ Median of Two Sorted Arrays
- ☐ Find First and Last Position of Element in Sorted Array
- ☐ Capacity To Ship Packages Within D Days
- ☐ Split Array Largest Sum
- ☐ Find Peak Element
- ☐ Search Insert Position
- ☐ Sqrt(x)
- ☐ Kth Smallest Element in a Sorted Matrix

6. Sliding Window

Core pattern: contiguous subarray/substring optimization, fixed/variable window

- ☐ Best Time to Buy and Sell Stock
- ☐ Longest Substring Without Repeating Characters
- ☐ Longest Repeating Character Replacement
- ☐ Permutation in String
- ☐ Minimum Window Substring
- ☐ Sliding Window Maximum
- ☐ Maximum Sum Subarray of Size K
- ☐ Fruit Into Baskets
- ☐ Find All Anagrams in a String
- ☐ Longest Subarray of 1's After Deleting One Element
- ☐ Minimum Size Subarray Sum
- ☐ Subarrays with K Different Integers
- ☐ Max Consecutive Ones III

7. Trees

Core pattern: DFS (preorder/inorder/postorder), BFS, recursion, tree construction

- ☐ Invert Binary Tree
- ☐ Maximum Depth of Binary Tree
- ☐ Diameter of Binary Tree
- ☐ Balanced Binary Tree
- ☐ Same Tree
- ☐ Subtree of Another Tree
- ☐ Lowest Common Ancestor of a BST
- ☐ Binary Tree Level Order Traversal
- ☐ Binary Tree Right Side View
- ☐ Count Good Nodes in Binary Tree
- ☐ Validate Binary Search Tree
- ☐ Kth Smallest Element in a BST
- ☐ Construct Binary Tree from Preorder and Inorder Traversal
- ☐ Binary Tree Maximum Path Sum
- ☐ Serialize and Deserialize Binary Tree
- ☐ Path Sum II
- ☐ Symmetric Tree
- ☐ Flatten Binary Tree to Linked List
- ☐ Populating Next Right Pointers in Each Node
- ☐ Vertical Order Traversal of a Binary Tree

8. Tries

Core pattern: prefix tree — string prefix matching, autocomplete-style problems

- ☐ Implement Trie (Prefix Tree)
- ☐ Design Add and Search Words Data Structure
- ☐ Word Search II
- ☐ Replace Words
- ☐ Maximum XOR of Two Numbers in an Array
- ☐ Longest Word in Dictionary
- ☐ Design Search Autocomplete System
- ☐ Palindrome Pairs
- ☐ Stream of Characters

9. Heap / Priority Queue

Core pattern: min-heap/max-heap for top-k, scheduling, running median

- ☐ Kth Largest Element in a Stream
- ☐ Last Stone Weight
- ☐ K Closest Points to Origin
- ☐ Kth Largest Element in an Array
- ☐ Task Scheduler
- ☐ Design Twitter
- ☐ Find Median from Data Stream
- ☐ Merge K Sorted Lists
- ☐ Top K Frequent Elements
- ☐ Reorganize String
- ☐ Meeting Rooms II
- ☐ Minimum Cost to Connect Sticks
- ☐ Single-Threaded CPU
- ☐ IPO (Maximize Capital)
- ☐ Process Tasks Using Servers

10. Backtracking

Core pattern: exploring a decision tree, pruning — combination/permutation/constraint satisfaction

- ☐ Subsets
- ☐ Combination Sum
- ☐ Combination Sum II
- ☐ Permutations
- ☐ Permutations II
- ☐ Subsets II
- ☐ Word Search
- ☐ Palindrome Partitioning
- ☐ Letter Combinations of a Phone Number
- ☐ N-Queens
- ☐ N-Queens II
- ☐ Sudoku Solver
- ☐ Combination Sum III
- ☐ Generate Parentheses
- ☐ Restore IP Addresses
- ☐ Word Break II
- ☐ Matchsticks to Square

Suggested Timeline (for fast prep)

Week	Topic	Goal
1	Arrays + Two Pointers	25–30 problems, understand patterns rather than memorize
2	Stack + Linked List	25 problems
3	Binary Search + Sliding Window	25 problems
4	Trees	20 problems (this will take the most time)
5	Tries + Heap	20 problems
6	Backtracking + Full Review	Revisit all weak spots + timed mock interview

Tip: Try each problem yourself for 25–30 minutes first. If stuck, look at a hint, not the full solution. After solving each problem, write a one-line summary of your approach in your own words — this builds pattern recognition.

Let me know if you'd like a detailed study sheet for any specific topic next, with approach hints, time/space complexity, and pattern summaries for each problem — same as the Arrays chapter.